MECHATRONICS ENGINEERING PROJECT

SMART INTELLIGENT SORTING SYSTEM

AI-Driven Industrial Inspection & Autonomous Robotic Sorting

AI Project Team | Supervisor: Prof. Amro Shafik

Department of Mechatronics Engineering | 2026

MEET THE TEAM

- Mohamed Abdeltawab
- Mahmoud Mohamed shamek
- Omar Mostafa Shokran
- Omar Mahmoud Metwally

SUPERVISOR

Prof. Amro Shafik
Eng. Mohamed Nassier

PROBLEM STATEMENT

THE CHALLENGE

Manual Inspection is Broken

Human Error

Up to 20% defect miss rate in manual visual inspection

Slow Process

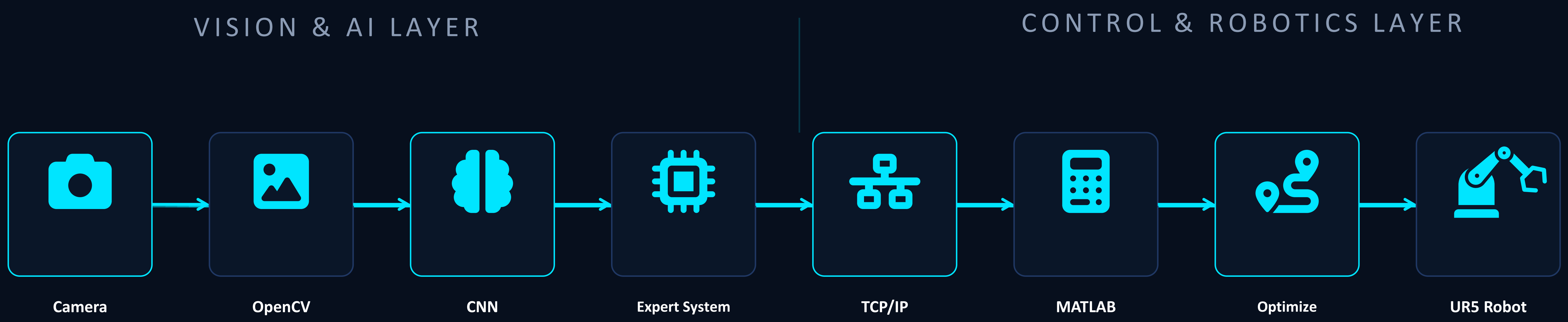
Quality control bottlenecks reduce throughput by 30%

No Automation

Lack of integrated AI-driven systems for real-time defect detection and robotic sorting

Solution: AI + Computer Vision + Autonomous Robotics

SYSTEM ARCHITECTURE

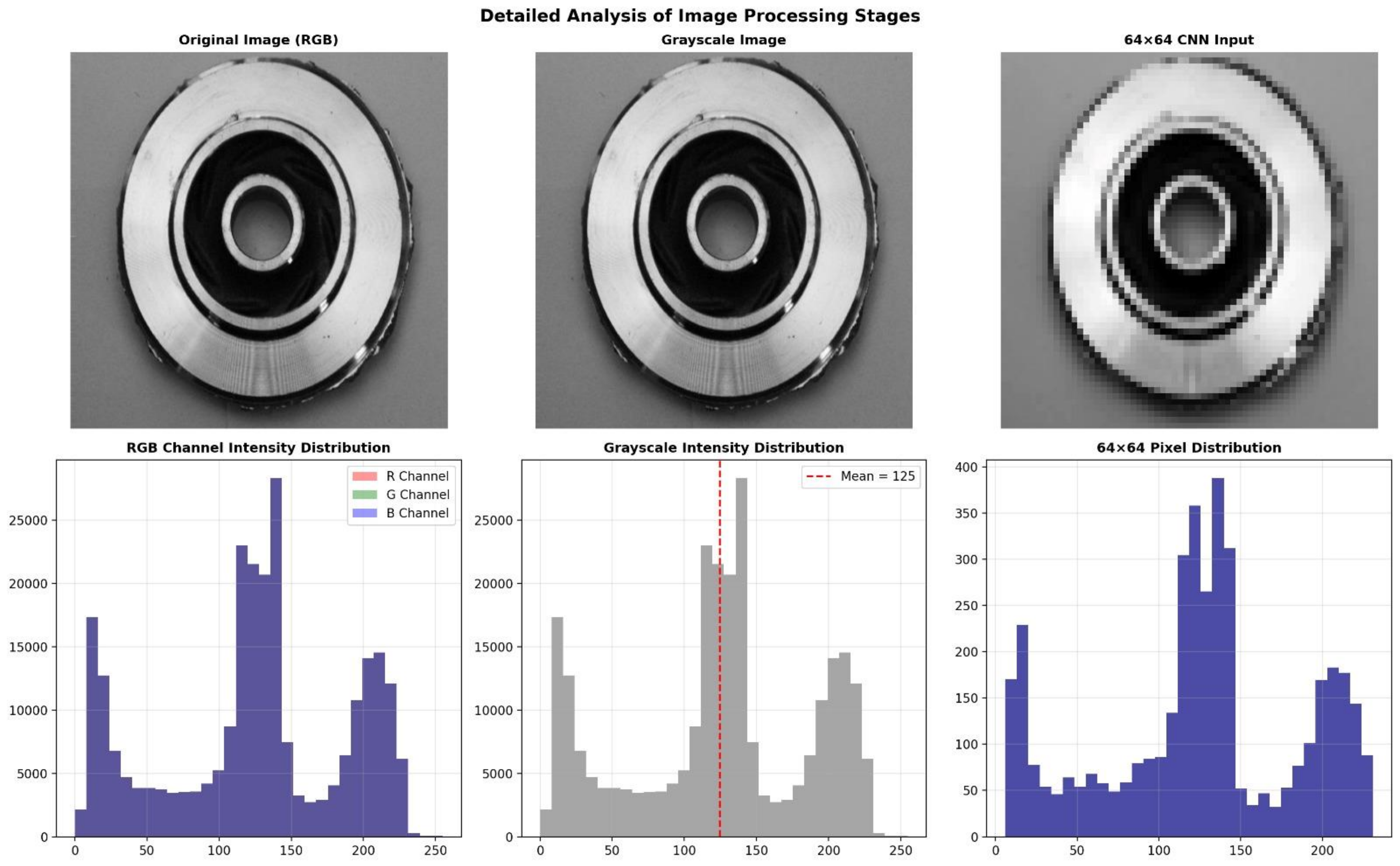


End-to-end integration from image capture through AI classification to autonomous robotic sorting

Seamless communication between Python vision pipeline and MATLAB robotics controller via TCP/IP

VISION SYSTEM

OpenCV Image Preprocessing Pipeline



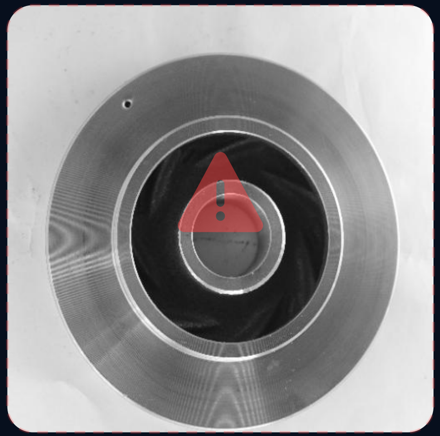
1. Capture | 2. Grayscale Conversion | 3. Gaussian Blur | 4. Resize to 64×64 | 5. Normalize

DATASET

Industrial Casting Product Images

DEFECTIVE SAMPLES


Defect Type A

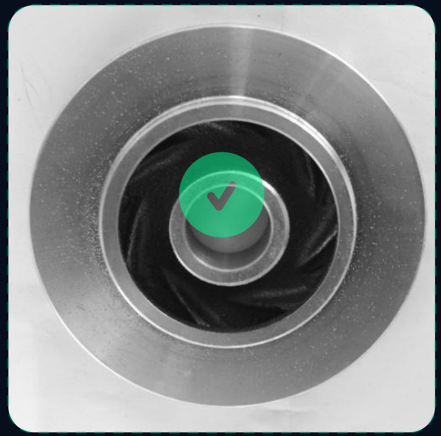

Defect Type B


Defect Type C

Scrap: porosity, cracks, sand holes, shrinkage, and surface defects

NON-DEFECTIVE SAMPLES


Good Sample A


Good Sample B


Good Sample C

Pass: clean surface, correct dimensions, no visible defects or anomalies

2,400+	1,200	1,200	64×64	80/20
Total Images	Defective	Non-Defective	Input Resolution	Train/Test Split

CNN ARCHITECTURE

PyTorch | TorchVision | NumPy

INPUT
64 × 64 × 1
Grayscale image tensor batch_size=32

→

CONV + ReLU
32 filters, 3×3
Feature extraction + BatchNorm

→

POOLING
MaxPool 2×2
Spatial downsampling 32 ⟶ 16

→

FULLY CONNECTED
128 ⟶ 64 ⟶ 2
Flatten + Dropout(0.5) + Linear

→

OUTPUT
Softmax, 2 classes
OK vs Defective + Sigmoid

Architecture Details

2 Conv blocks with BatchNorm

MaxPooling after each block

2 FC layers with Dropout

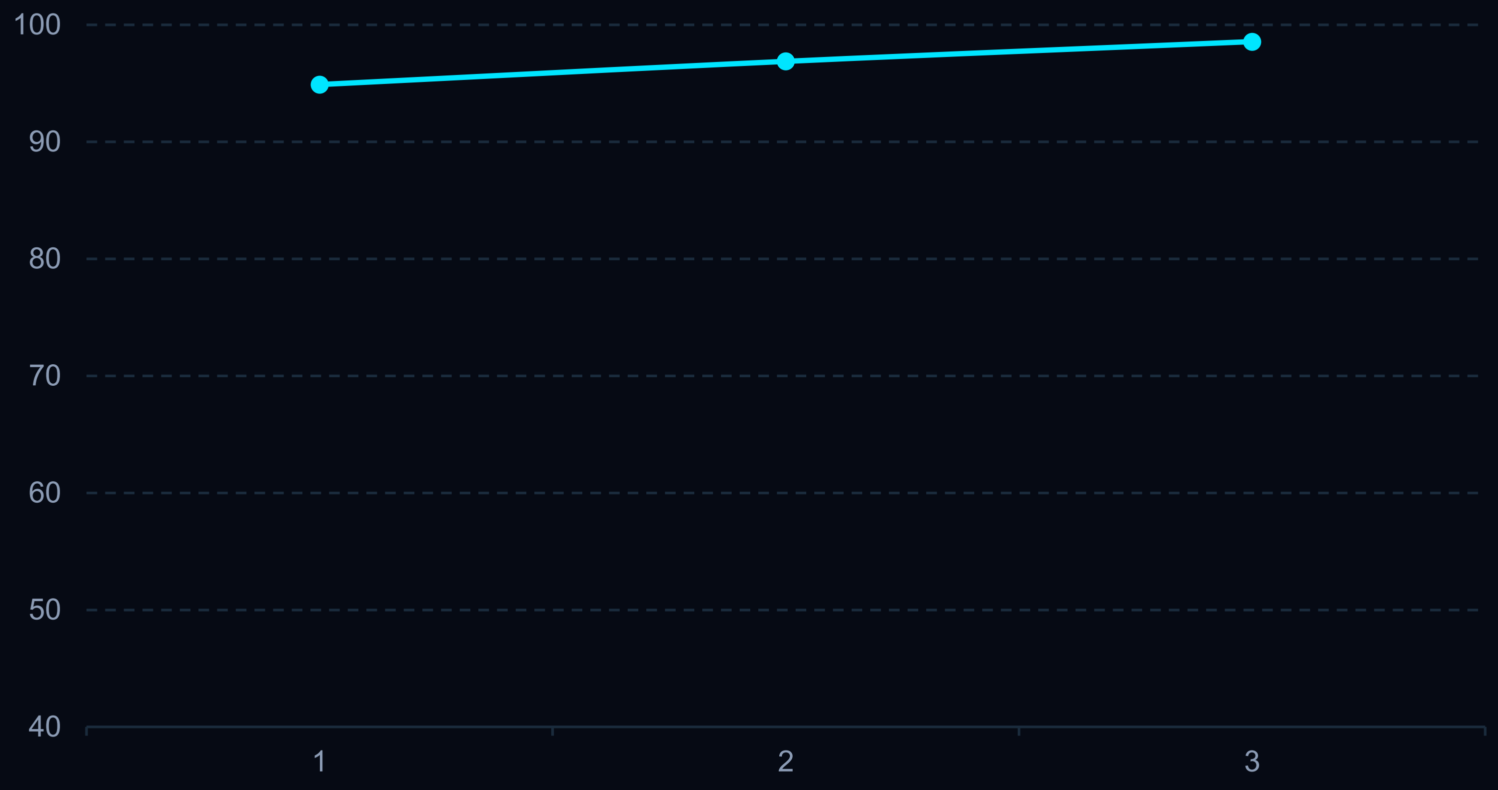
Adam optimizer, lr=0.001

CrossEntropyLoss

50 epochs, early stopping

CNN TRAINING

98.57%	0.0531	3	~12m
TEST ACCURACY	FINAL LOSS	EPOCHS	TRAINING TIME



ACCURACY OVER EPOCHS

LOSS REDUCTION OVER EPOCHS

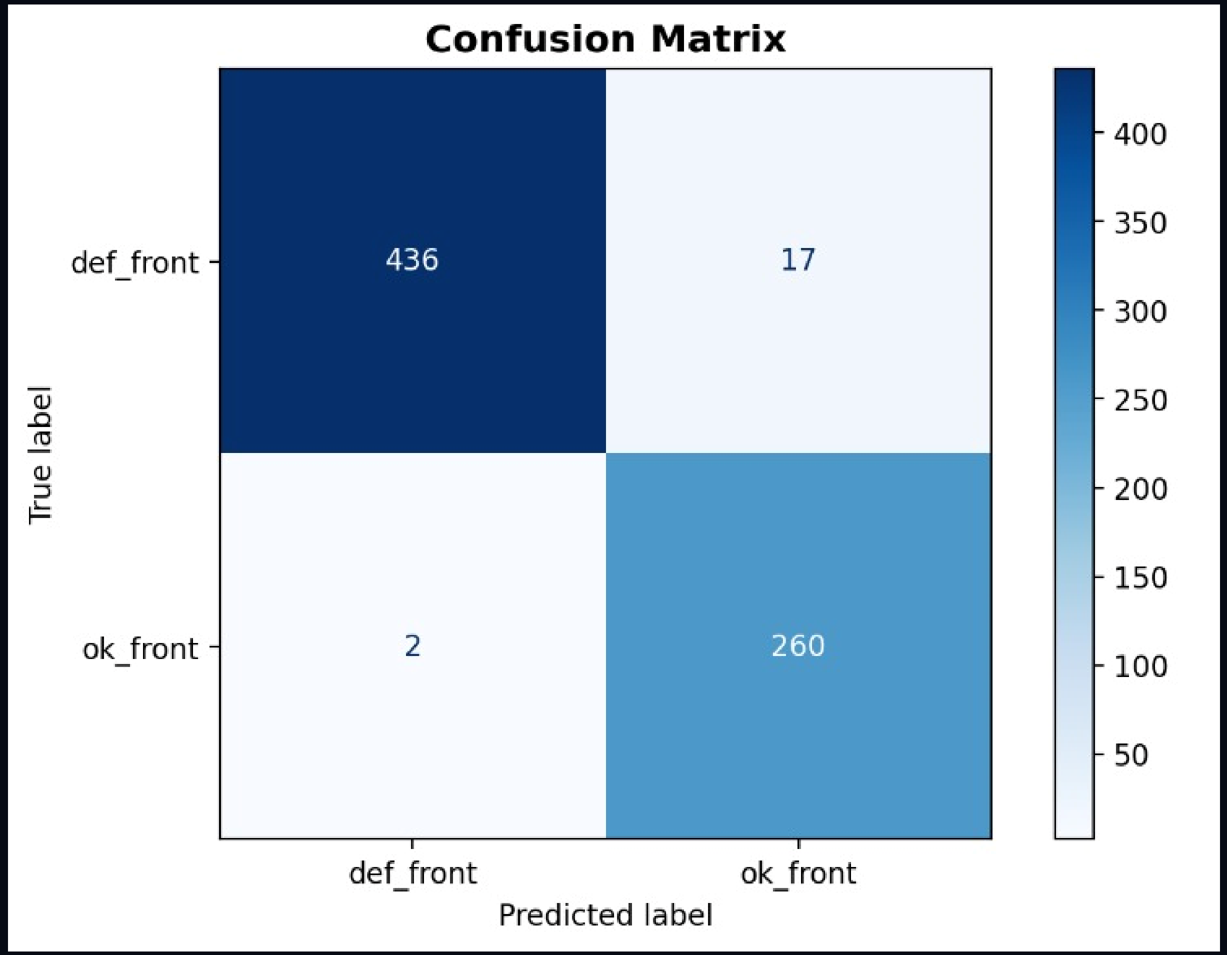
MODEL EVALUATION

CONFUSION MATRIX

	Pred: OK	Pred: Defect
Actual: OK	260	2
Actual: Defect	17	436
Total	277	438

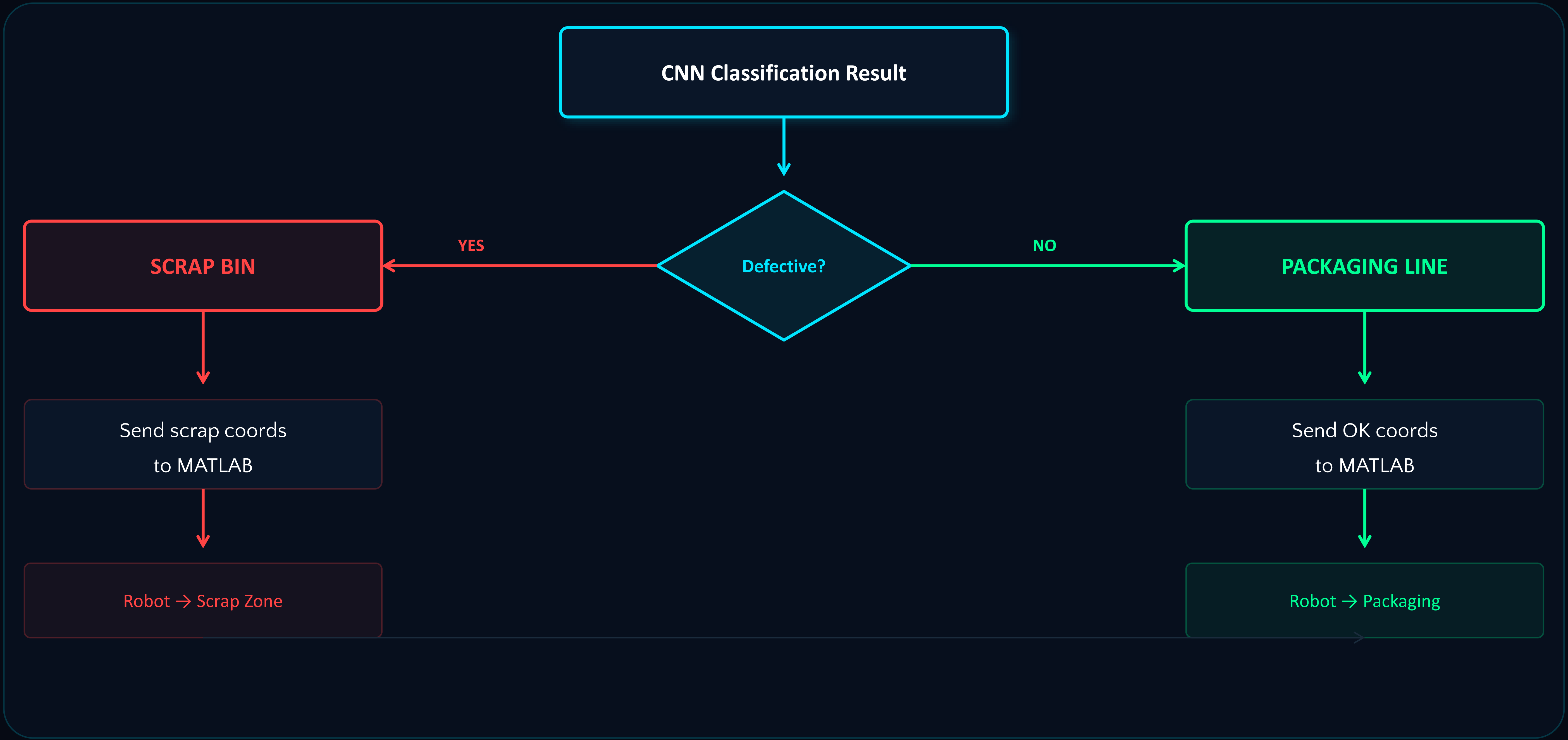
Test set: 1,200 images | True Negatives: 568 | True Positives: 584

False Positives: 32 | False Negatives: 16



EXPERT SYSTEM

Rule-Based Decision Engine



TCP/IP COMMUNICATION

Python Client ↔ MATLAB Server Bridge

PYTHON CLIENT

- Captures image from camera
- Preprocesses with OpenCV
- Runs CNN inference
- Expert system decision
- Extracts object coordinates
- Creates TCP socket client
- Sends (x, y, class) packet

```
socket.send(coords.encode())
```

x,y,cls

x,y,cls

TCP/IP

MATLAB SERVER

- Opens TCP/IP socket
- Listens on port 65432
- Receives coordinate data
- Parses (x, y, class) packet
- Computes inverse kinematics
- Plans optimized trajectory
- Sends to CoppeliaSim

```
fopen(tcpip('0.0.0.0',55000))
```

PROTOCOL DETAILS

Protocol: TCP/IP

Port: 65432

Format: JSON string

Payload:

{"x": 120,

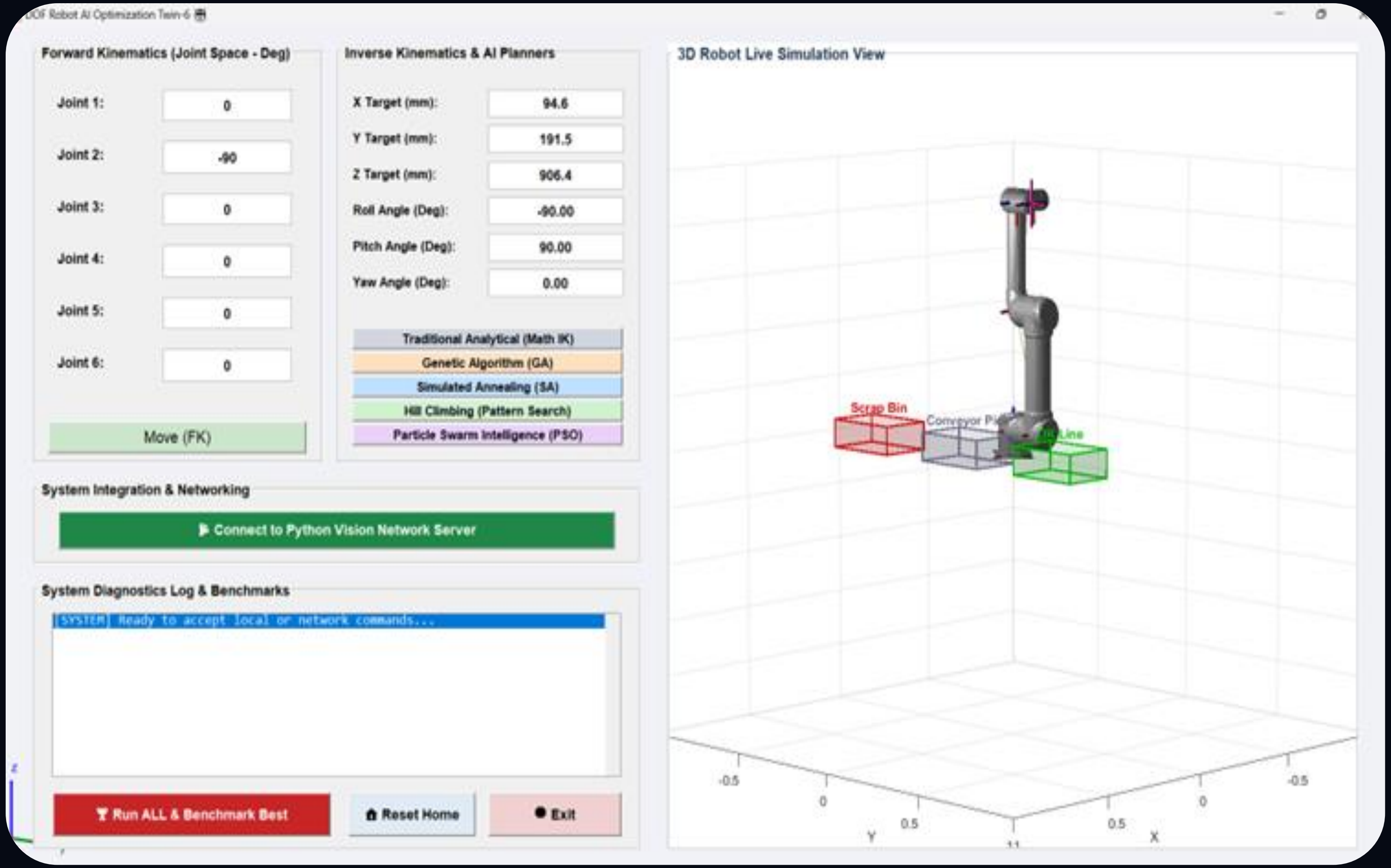
"y": 85,

"class": "ok"}

Latency: < 50ms

MATLAB DIGITAL TWIN

Real-Time Robot Visualization & Control



MATLAB GUI PREVIEW

DIGITAL TWIN FEATURES

3D Robot Visualization
Real-time UR5 model with joint angles

Joint Control Panels
Individual joint angle sliders (J1–J6)

Trajectory Plotting
End-effector path visualization

TCP/IP Listener
Receives coordinates from Python

CoppeliaSim Sync
Real-time simulation synchronization

INVERSE KINEMATICS & PATH PLANNING

FORWARD KINEMATICS

$$T = A_1(\theta_1) \cdot A_2(\theta_2) \cdots A_6(\theta_6)$$

Given joint angles $\theta_1 \ldots \theta_6$, compute end-effector position and orientation in Cartesian space

Using Denavit-Hartenberg (DH) parameters for UR5 6-DOF manipulator

INVERSE KINEMATICS

$$\theta = f^{-1}(x, y, z, \alpha, \beta, \gamma)$$

Given target position (x,y,z) and orientation, solve for joint angles $\theta_1 \ldots \theta_6$

Analytical solution + Jacobian-based numerical refinement for accuracy

TRAJECTORY GENERATION

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

Cubic Polynomial Interpolation

$$\dot{q}(t) = a_1 + 2a_2 t + 3a_3 t^2$$

Velocity Profile (Smooth Motion)

$$\ddot{q}(t) = 2a_2 + 6a_3 t$$

Acceleration

Boundary conditions: position, velocity, and acceleration constraints at start/end points for smooth motion

OPTIMIZATION ALGORITHMS

Robot Path Optimization — Finding the Optimal Trajectory

01

Genetic Algorithm

GA

Selection, crossover, mutation

Population-based search

Global exploration

Generational evolution

02

Particle Swarm

PSO

Swarm intelligence

Velocity-position update

Fast convergence

Local + global best

03

Simulated Annealing

SA

Temperature-based search

Probabilistic acceptance

Escapes local minima

Cooling schedule

04

Hill Climbing

HC

Local search heuristic

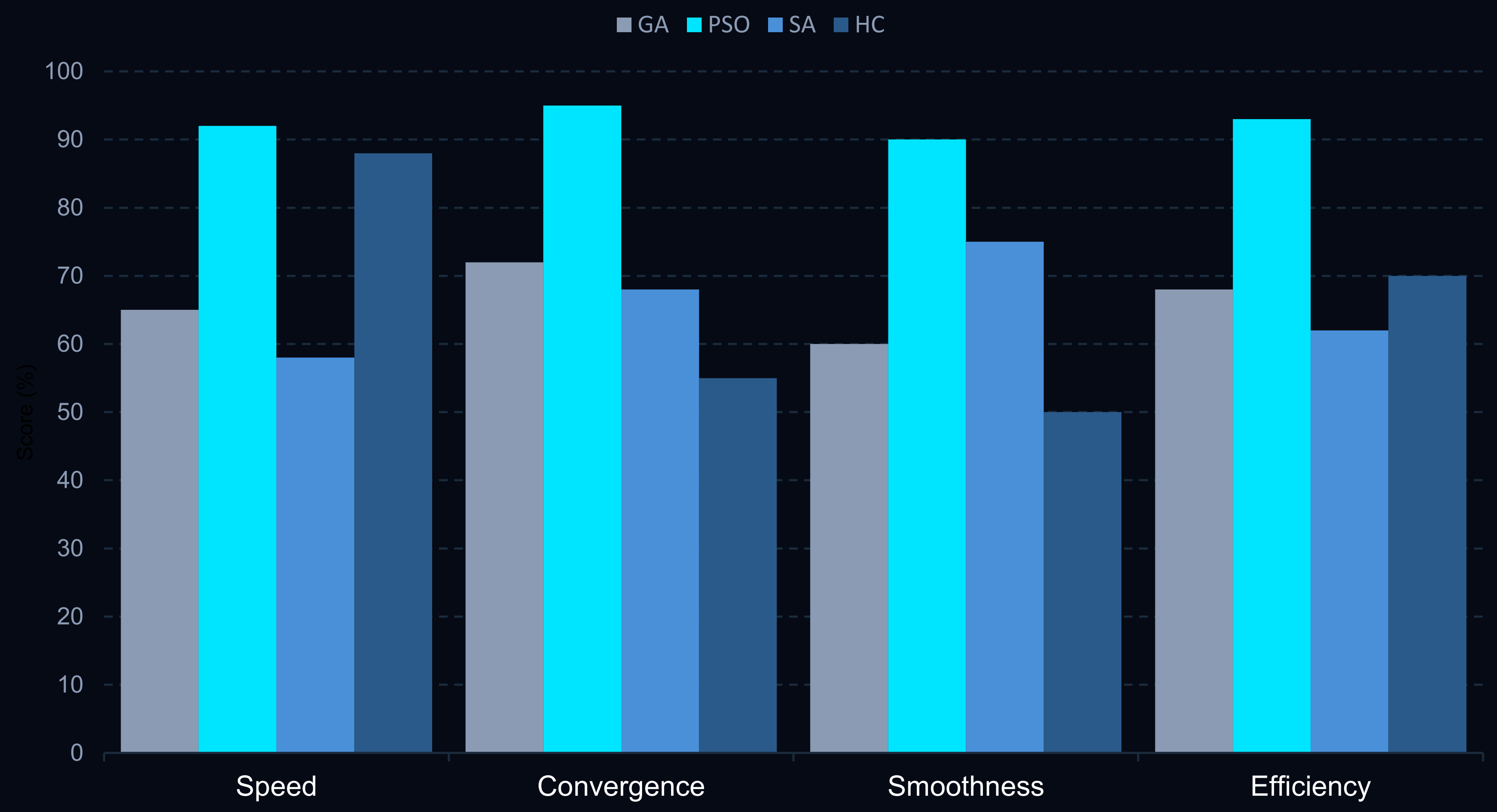
Greedy neighbor selection

Simple implementation

Prone to local optima

BENCHMARK RESULTS

PSO emerges as the best performer across all metrics — fastest convergence, smoothest paths, highest efficiency



WINNER: PSO

92.5%

AVERAGE SCORE

- Converges in ~40 iterations
- Path smoothness: 90%
- Computation time: 0.8s
- Collision-free trajectories

COPPELIASIM INTEGRATION

Real-Time Simulation Synchronization

MATLAB
IK + Trajectory
+ Optimization

Joint Angles

UR5 Robot
Physical Model
6-DOF Manipulator

Remote API

CoppeliaSim
3D Physics Simulation
Real-time Visualization

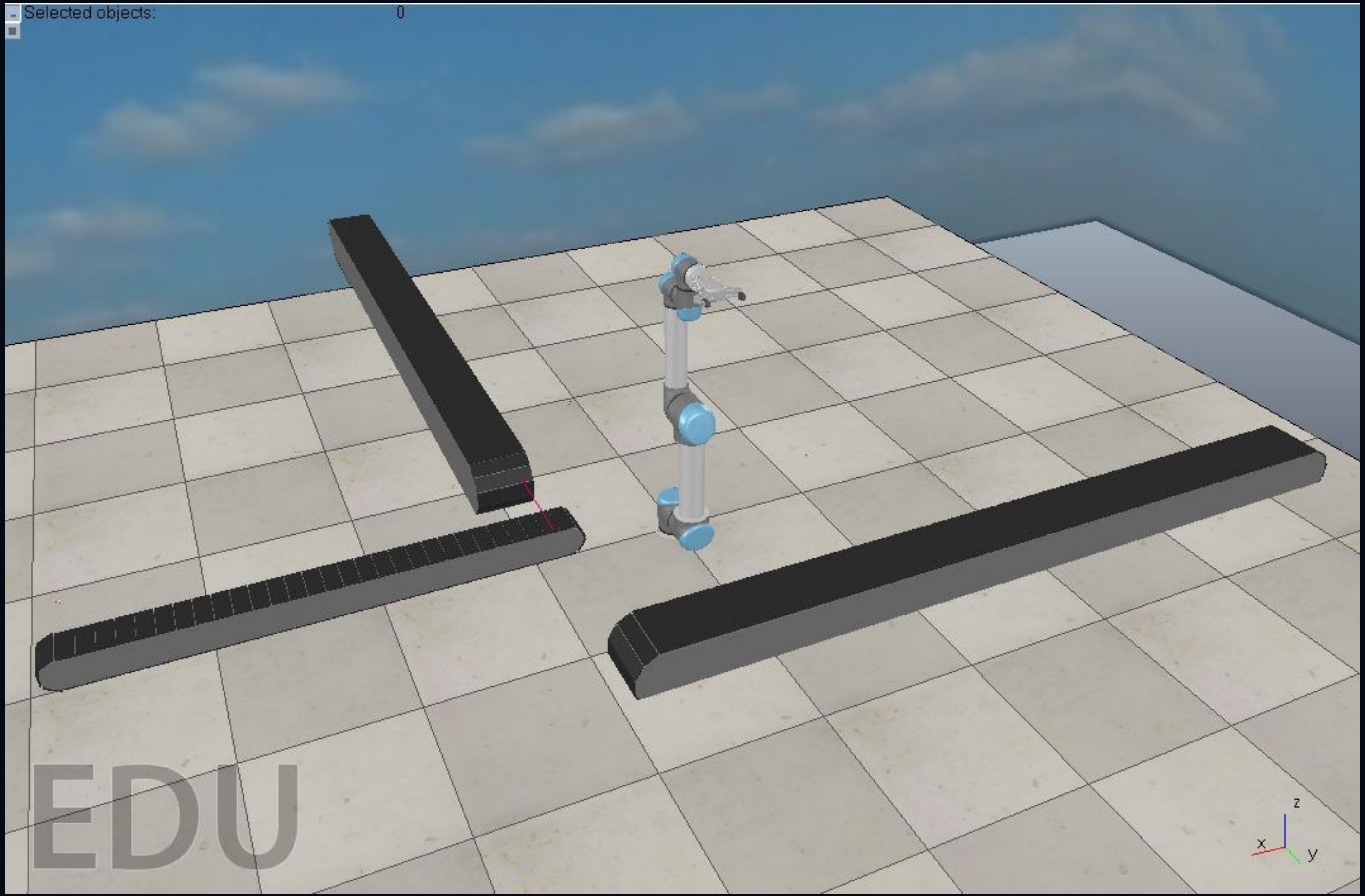
Position Feedback Loop

SYNCHRONIZATION FEATURES

- Real-time joint angle streaming at 50Hz
- Physics-based collision detection
- Visual feedback for trajectory validation
- MATLAB Remote API (legacy) protocol

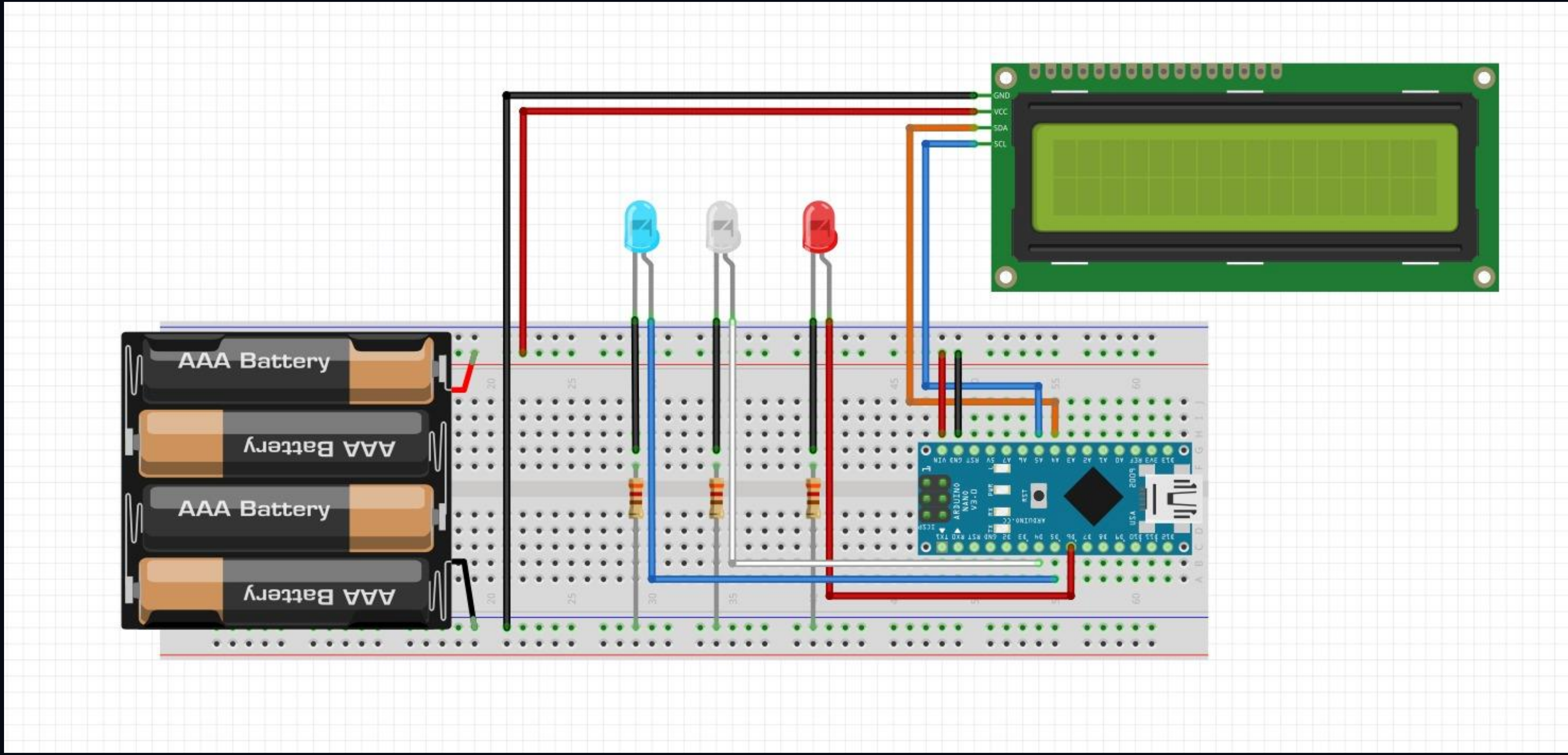
COPPELIASIM INTEGRATION

Real-Time Simulation Synchronization



Hardware INTEGRATION

Real-Time hardware



FINAL AUTONOMOUS SIMULATION

LIVE DEMO WORKFLOW

AI Detects

Camera captures object image from conveyor

→

CNN Classifies

Neural network identifies OK or defective

→

MATLAB Receives

TCP/IP transfers coordinates to controller

→

Robot Moves

UR5 executes optimized trajectory

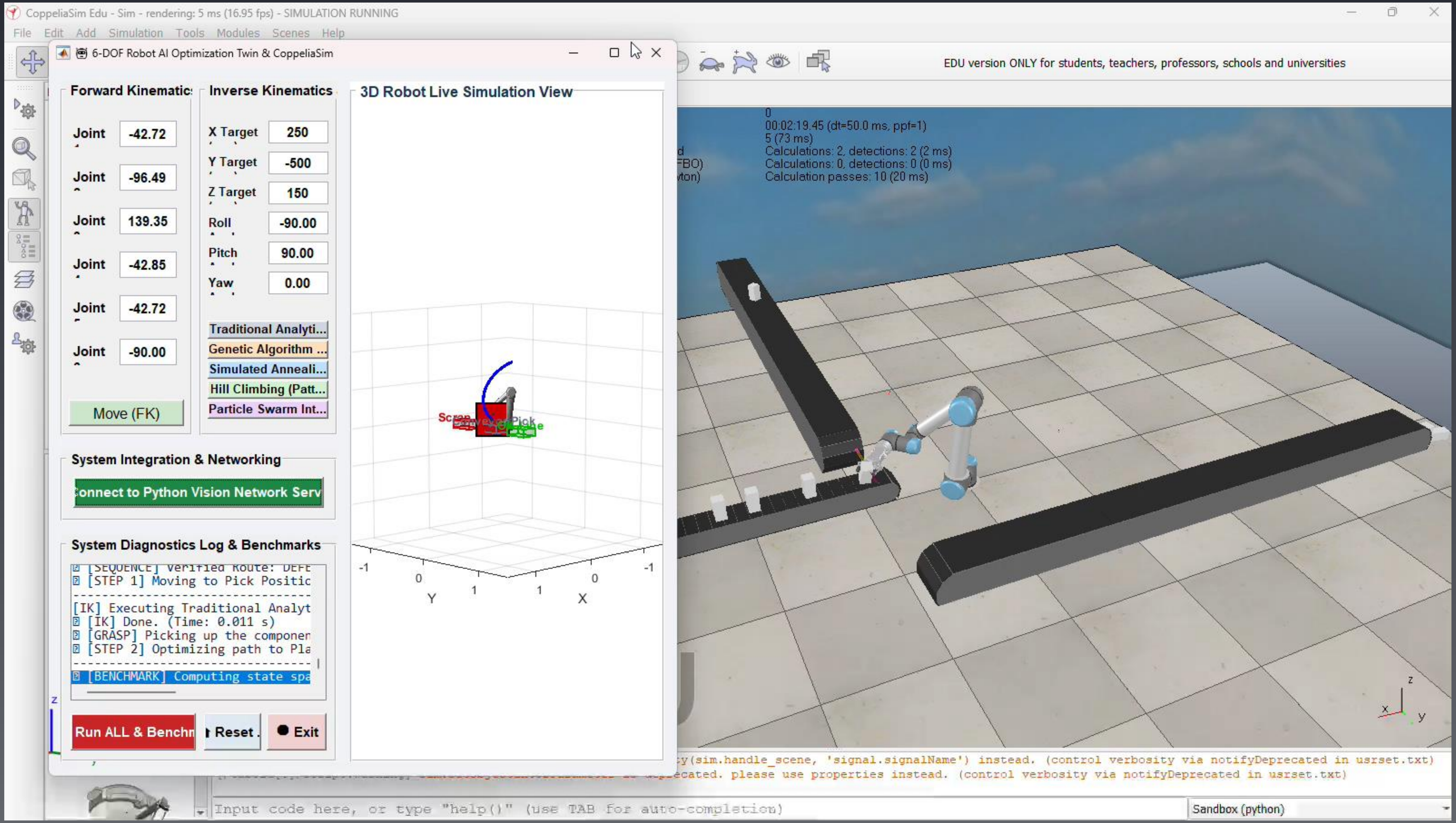
→

SORTED

< 2.5s	92.04%	100%	PSO	Fully
Cycle Time	Classification	Sorting Success	Best Optimizer	Autonomous

FINAL AUTONOMOUS SIMULATION

LIVE DEMO WORKFLOW



ADVANTAGES

AI Automation

Deep learning-powered defect detection eliminates human error and subjective judgment

Smart Manufacturing

Industry 4.0 ready with intelligent quality control and automated sorting workflows

Real-Time Robotics

Sub-3-second response from detection to physical sorting action

Industry 4.0

Full digital integration with networked sensors, AI, and robotic actuators

Digital Twin

MATLAB-based virtual replica for simulation, testing, and validation before deployment

Autonomous Decisions

Expert system enables fully autonomous sorting without human intervention

CONCLUSION

TOWARD AUTONOMOUS

INDUSTRY 4.0 MANUFACTURING

This project demonstrates a complete AI-driven autonomous sorting pipeline integrating computer vision, deep learning, expert systems, and robotics — achieving 92.04% accuracy with < 2.5s end-to-end cycle time.

- CNN Defect Detection
- Optimization technics
- TCP/IP Integration

FUTURE WORK

- **Real Industrial Robot Deployment**
  Physical UR5 on factory floor
- **Real Conveyor Belt System**
  Continuous product feed integration
- **Real-Time Camera Array**
  Multi-angle inspection setup
- **IIoT Integration**
  Industrial IoT sensor network
- **Cloud Monitoring Dashboard**
  Remote analytics and control
- **Advanced CNN Models**
  ResNet, EfficientNet for higher accuracy

THANK YOU

QUESTIONS & ANSWERS

Smart Intelligent Sorting System | Mechatronics Engineering | 2026

Team: [Member 1] | [Member 2] | [Member 3]

Supervisor: Prof. [Supervisor Name]